

ULK POLYTECHNIC INSTITUTE

Integrated Situation Technical Report

Cryptography & Network Security (ETTCS801)

Roll Number: 4202670059 / Lecturer: Isaac TUMWINE

September 29, 2026

Abstract

This technical report details the implementation of security measures designed to address operational and network vulnerabilities identified at ULK Polytechnic Institute. The scope of work encompasses a comprehensive risk assessment, the creation of a custom Python security toolkit employing AES (Fernet) symmetric encryption and SHA-256 integrity verification, secure git version control setup, and the deployment of host-based firewall rules using `iptables`. Empirical testing validates that sensitive student records remain confidential, file integrity is verifiable, and unauthorized guest access to central server assets is completely restricted.

1 Risk Assessment and Defense Strategy

An initial security review revealed several critical deficiencies across the central campus network, including unencrypted data transfers between campuses, unisolated guest subnet access, and weak password practices.

1.1 Asset and Vulnerability Identification

1. Student Records Database & Archives:

- *Vulnerability:* Unencrypted file storage and cross-campus transfer over public links.
- *Consequences:* Cleartext eavesdropping, unauthorized data modification, and severe compliance violations regarding student privacy.

2. Central Server Infrastructure:

- *Vulnerability:* Direct Guest network access to internal server subnets without host filtering boundaries.
- *Consequences:* Network reconnaissance, port scanning, unauthorized service abuse, and lateral movement.

3. Administrative & Staff Accounts:

- *Vulnerability:* Weak password policies and outdated software packages on endpoints.
- *Consequences:* Increased exposure to brute-force authentication attacks and local privilege escalation.

1.2 Risk Evaluation & Priority Ranking

Table 1: Risk Evaluation Matrix

Risk ID	Description	Likelihood	Impact	Priority Rank
R1	Unencrypted Data Storage & Transmission	High	High	Critical (Rank 1)
R2	Direct Guest Network Access to Central Server	High	Medium	High (Rank 2)
R3	Weak Staff Passwords & Credential Compromise	Medium	High	Medium (Rank 3)

Ranking Justification:

- **Rank 1 (Unencrypted Data):** Cleartext transfers across active inter-campus channels present immediate exposure to passive sniffing, posing the highest risk to confidentiality.
- **Rank 2 (Guest Access):** Open subnet routing allows unauthenticated devices to directly query server ports without network restriction.
- **Rank 3 (Weak Passwords):** Credential harvesting requires active targeted interaction, making it a secondary exposure compared to open unencrypted network channels.

1.3 Recommended Controls

1. **Data Security Control:** Enforce symmetric encryption (AES via Fernet) for stored files and transit assets combined with SHA-256 hash checks.
2. **Network Filtering Control:** Configure `iptables` rules to explicitly block all incoming traffic originating from the untrusted Guest subnet (192.168.20.0/24).
3. **Identity & Access Control:** Enforce Multi-Factor Authentication (MFA), password complexity policies, and scheduled software updates.

2 Cryptographic Toolkit Implementation & Verification

To enforce confidentiality and integrity, a Python tool (`security_toolkit.py`) was developed utilizing the Python `cryptography` library and standard `hashlib` module.

2.1 Core Architecture

The toolkit handles AES symmetric encryption using the **Fernet** specification (which applies AES-128 in CBC mode with PKCS7 padding and HMAC authentication) and provides SHA-256 hashing functions. Key management ensures that the encryption key (`secret.key`) and generated encryption payloads are excluded from version control via a tailored `.gitignore` configuration.

Version Control Protection (`.gitignore`): To comply with security best practices and project requirements, sensitive runtime files, keys, and auxiliary build artifacts are excluded using standard pattern rules in `.gitignore`:

Listing 1: Repository Security Rules in `.gitignore`

```
# Encryption Keys & Secrets (CRITICAL REQUIREMENT)
secret.key
*.key
*.pem
*.pub

# Local Test Data & Encryption Outputs
*.enc
decrypted_*
sample_student_record*

# LaTeX Auxiliary & Build Files
*.aux
*.log
*.out
```

2.2 Program Implementation

Listing 2: Core Functions of `security_toolkit.py`

```
def encrypt_file(file_path):
    key = generate_or_load_key()
    fernet = Fernet(key)
    with open(file_path, "rb") as f:
        data = f.read()
    encrypted_data = fernet.encrypt(data)
    with open(file_path + ".enc", "wb") as f:
        f.write(encrypted_data)

def calculate_sha256(file_path):
    sha256_hash = hashlib.sha256()
    with open(file_path, "rb") as f:
        for byte_block in iter(lambda: f.read(4096), b''):
```

```
        sha256_hash.update(byte_block)
    return sha256_hash.hexdigest()
```

2.3 Test Evidence & Verification Workflows

Testing was executed against a target sample record file:

1. Encryption & SHA-256 Hash Generation:

```
$ python3 security_toolkit.py --encrypt sample.txt
[+] Original SHA-256 Hash:
    ↪ e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855
    ↪
[+] File encrypted successfully. Output saved to: 'sample.txt.
    ↪ enc'
```

2. Decryption & Integrity Hash Match:

```
$ python3 security_toolkit.py --decrypt sample.txt.enc
[+] File decrypted successfully. Output saved to: 'sample.txt'
[+] Decrypted File SHA-256 Hash:
    ↪ e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855
    ↪
```

3. Tamper Detection Verification:

```
$ python3 security_toolkit.py --verify sample.txt
    ↪ INVALID_HASH_STRING
[+] Current SHA-256 Hash:
    ↪ e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855
    ↪
[+] Expected SHA-256 Hash: INVALID_HASH_STRING
[!] INTEGRITY VERIFICATION: FAILED (File has been altered or
    ↪ tampered with!)
```

3 Network Traffic Filtering

Network filtering controls were implemented in the laboratory environment using Linux `iptables` on the Central Server (192.168.10.100) to isolate server resources from untrusted subnets.

3.1 Target Topology

- **Central Server IP:** 192.168.10.100
- **Staff Subnet (Authorized):** 192.168.10.0/24 (Permitted on HTTP Port 80)
- **Guest Subnet (Untrusted):** 192.168.20.0/24 (Strictly Isolated)

3.2 Firewall Rule Architecture

The following rules establish a default-drop policy and explicitly configure access controls:

Listing 3: iptables Rules Configuration

```
# Flush existing rules and set default-deny policies
sudo iptables -F
sudo iptables -P INPUT DROP
sudo iptables -P FORWARD DROP

# Allow loopback and established connections
sudo iptables -A INPUT -i lo -j ACCEPT
sudo iptables -A INPUT -m state --state ESTABLISHED,RELATED -j
    ↪ ACCEPT

# Block untrusted Guest network traffic
sudo iptables -A INPUT -s 192.168.20.0/24 -j DROP

# Permit HTTP access specifically from Staff network
sudo iptables -A INPUT -s 192.168.10.0/24 -p tcp --dport 80 -j
    ↪ ACCEPT

# Explicitly drop any remaining HTTP requests
sudo iptables -A INPUT -p tcp --dport 80 -j DROP
```

3.3 Empirical Test Execution Log

Empirical verification was conducted using netcat (nc):

Table 2: Firewall Verification Empirical Results

Test Case	Command Executed	Observed Output	Verdict
Test 1: Permitted Staff HTTP	nc -zv -s 192.168.10.15 192.168.10.100 80	Connection to 192.168.10.100 80 port [tcp/http] succeeded!	PASS
Test 2: Blocked Guest Access	nc -zv -w 3 -s 192.168.20.45 192.168.10.100 80	nc: connect to 192.168.10.100 port 80 timed out: Operation in progress	PASS
Test 3: Blocked Service (SSH Port 22)	nc -zv -w 3 -s 192.168.10.15 192.168.10.100 22	nc: connect to 192.168.10.100 port 22 timed out: Operation in progress	PASS

4 Conclusion and Submission Details

This project successfully addresses the security weaknesses identified in the ULK Polytechnic Institute infrastructure. Symmetric AES encryption and SHA-256 checksum mechanisms protect sensitive student records against unauthorized disclosure and tampering. Furthermore, default-deny `iptables` firewall configurations effectively isolate the records server from untrusted guest networks while maintaining access for authorized staff.

4.1 Repository Information

All project files—including source code, test execution logs, risk assessments, the security-focused `.gitignore` configuration, and LaTeX project sources—have been committed to the GitHub repository below:

GitHub Repository URL:

<https://github.com/ugirashebuja-moise/cryptography-network-security-exam.git>